

BÁO CÁO BÀI TẬP LỚN: HỆ THỐNG ĐẤU GIÁ TRỰC TUYẾN (MiniBoom)

- **Ngành:** Công nghệ Thông tin - Trường Đại học Công nghệ (UET) - ĐHQGHN
 - **Môn học:** Lập trình nâng cao
 - **Dự án:** MiniBoom (Desktop App Client-Server Realtime)
 - **Kiến trúc:** Multi-module Maven (Java 25, JavaFX 21, SQLite WAL)
 - **Repository:** <https://github.com/Trumkhungbo/baitaplon>
-

1. MỤC TIÊU VÀ PHẠM VI THỰC HIỆN

1.1. Mục tiêu dự án

Dự án MiniBoom được xây dựng nhằm mô phỏng nền tảng đấu giá tài sản trực tuyến hoạt động theo mô hình Client-Server Realtime. Mục tiêu cốt lõi tập trung giải quyết các bài toán kinh điển trong hệ thống phân tán:

- Đảm bảo tính đồng bộ dữ liệu thời gian thực (Real-time) giữa hàng loạt máy trạm.
- Xử lý đồng thời (Concurrency) triệt tiêu lỗi Race Condition khi nhiều người cùng đặt giá.
- Tự động hóa quy trình nghiệp vụ (Auto-bid, Anti-snipe) bằng các luồng chạy ngầm để đảm bảo tính công bằng tuyệt đối.

1.2. Phạm vi thực hiện

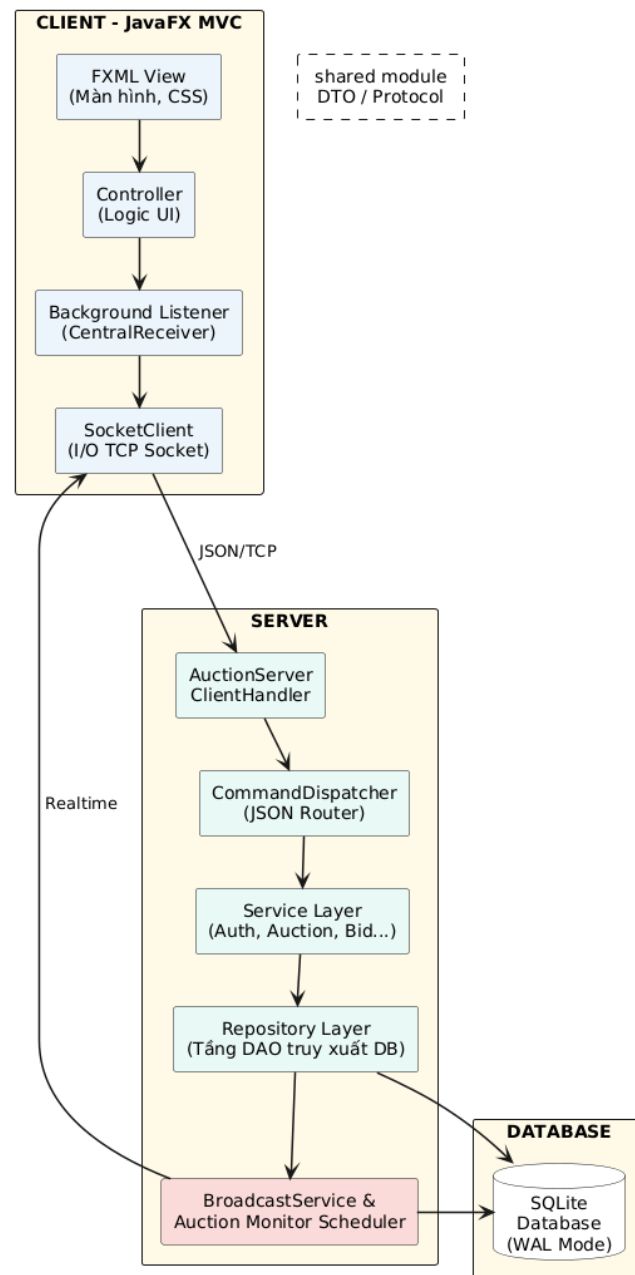
- **Kiến trúc Zero-Config:** Triển khai độc lập ("Cắm là chạy") thông qua CSDL nhúng SQLite thay vì phải cài đặt Server cấu hình phức tạp.
 - **Phân quyền linh hoạt:** Áp dụng RBAC cho 3 vai trò: ADMIN (Quản trị), SELLER (Người bán) và BIDDER (Người mua). Người dùng có thể vừa đăng bán vừa đặt giá bằng cùng một tài khoản.
 - **Nghiệp vụ khép kín:** Hỗ trợ vòng đời phiên đấu giá nghiêm ngặt, đặt giá thủ công, ủy quyền đấu giá tự động, kiểm duyệt từ Admin và thanh toán ví điện tử hậu kiểm.
-

2. KIẾN TRÚC TỔNG THỂ CỦA HỆ THỐNG

2.1. Sơ đồ kiến trúc tổng thể

Hệ thống MiniBoom bao gồm **3 module Maven**: **common** (DTO/Protocol), **client** (JavaFX MVC), và **server** (Service/Repository/Database). Việc phân rã module giúp

bảo vệ tính toàn vẹn của nghiệp vụ, ngăn chặn Client can thiệp trực tiếp vào Database.



2.2. Mô tả chi tiết theo sơ đồ kiến trúc

- **Module & MVC:** Module **common** chứa các lớp DTO, Payload và Entity dùng chung để chuẩn hóa giao thức; module **client** tách biệt hoàn toàn FXML (giao diện), CSS và Controller (xử lý logic UI); module **server** phân rã kiến trúc theo luồng Bộ định tuyến (CommandDispatcher) -> Nghiệp vụ (Service Layer) -> Truy xuất dữ liệu (DAO). Cách chia này giúp cô lập hoàn toàn truy cập cơ sở dữ liệu khỏi giao diện người dùng.
- **Realtime TCP:** Mạng giao tiếp thông qua **TCP Socket** (Port 888) sử dụng chuỗi định dạng **JSON** (đóng gói bằng thư viện Gson). Phía máy chủ,

BroadcastService chịu trách nhiệm bắt sự kiện và đẩy trực tiếp xuống **CentralReceiver** đang lắng nghe ngầm trên Background Thread tại **Client**.

- **Bidding Pipeline:** Các gói tin JSON đặt giá đi qua bộ định tuyến **CommandDispatcher**. Tại tầng cơ sở dữ liệu, hàm placeBid được bọc trong một **Transaction SQL nguyên tử**, đảm bảo thao tác cập nhật giá cao nhất và ghi nhận lịch sử vào bảng **bid_transactions** diễn ra đồng thời.

3. CHỨC NĂNG ĐẠT ĐƯỢC THEO BAREM ĐIỂM

Nhóm chức năng (Barem)	Hướng giải quyết & Kỹ thuật áp dụng	Lý do lựa chọn & Hiệu quả
Thiết kế OOP	- Kế thừa & Đa hình: Cha User rẽ nhánh Admin, Seller, Bidder. Cha Item rẽ nhánh Art, Electronics, Vehicle. - Đóng gói: Các thuộc tính bảo mật khóa private.	Tạo Domain Model mềm dẻo. Dễ mở rộng phân quyền và thêm loại tài sản mới mà không sửa core logic.
Design Pattern	- MVC: Tách rời FXML View và Java Controller. - Command Pattern: CommandDispatcher định tuyến API mạng. - Singleton & DAO: Cô lập truy xuất CSDL.	Kiến trúc Module hóa cao. Tách biệt UI/Logic. Dễ dàng bổ sung tính năng bằng cách implement interface.
User & Product	- Băm mật khẩu chuẩn PBKDF2 kèm Salt ngẫu nhiên. - Truyền tải file ảnh bằng chuỗi nhị phân Base64 qua TCP.	Chống brute-force tuyệt đối. Truyền ảnh qua TCP giúp duy trì hệ thống "Zero-Config" chỉ với 1 cổng mạng.
Đấu giá & Xử lý lỗi	- Pipeline độc lập: AuctionLifecycleService (vòng đời) và BidService (đặt giá). - Tự định nghĩa Exception, SQL Rollback khi lỗi.	Tách biệt side-effect tài chính và trạng thái. Server không commit nửa vời khi giao dịch gặp sự cố.

Nhóm chức năng (Barem)	Hướng giải quyết & Kỹ thuật áp dụng	Lý do lựa chọn & Hiệu quả
Đồng thời (Concurrency)	- Dùng khối synchronized(auction) khóa luồng tại Server. - SQLite kích hoạt WAL Mode (Write-Ahead Logging).	Chặn đứng triệt để lỗi <i>Race Condition</i> và <i>Database Locked</i> khi hàng chục người cùng bảo lệnh đặt giá.
Realtime Update	- TCP Socket 2 chiều duy trì liên tục. - Máy chủ dùng BroadcastService đẩy JSON Event. - Client dùng Platform.runLater() để cập nhật JavaFX.	Độ trễ thấp. Dữ liệu giá thầu hiển thị tức thời tạo tính cạnh tranh nghẹt thở trong phòng đấu giá.
Tích hợp & CI/CD	- Quản lý bằng Maven Multi-Module. Đóng gói Fat JAR. - Test tự động với JUnit 5, Mockito, TestFX. - Pipeline GitHub Actions.	Mã nguồn sạch. Tự động hóa kiểm thử. Xuất bản tệp JAR độc lập dễ triển khai ("Cắm là chạy").
Chức năng Nâng cao (Tùy chọn)	- Auto-Bidding: Thuật toán tự động tính bước giá chặn top 1. - Anti-sniping: Tự cộng 60s nếu có bid ở 30s cuối. - Atomic Checkout: Trừ/cộng tiền + lưu log trên 1 Transaction. - Live Chart: Cập nhật biểu đồ đường giá realtime.	Tăng tính công bằng, chống tool/bot. Giao dịch tài chính an toàn tuyệt đối (ACID). Trực quan hóa phiên đấu.

4. KỸ THUẬT NÂNG CAO VÀ ĐIỂM NỔI BẬT

4.1. Động cơ Đấu thầu Tự động Cạnh tranh chéo (Cross-Proxy Auto-Bidding)

Khác với việc người dùng phải online liên tục để gõ phím, hệ thống cho phép cài đặt mức giá trần tối đa (Max Bid). Khi có đối thủ trả giá thủ công, hàm `applyProxyAutoBidAfterManualBid()` sẽ lập tức phản đòn, tự động toán học hóa

bước giá tối thiểu (Increment) để đưa người ủy quyền lên vị trí dẫn đầu trong chóp mắt. Nếu có 2 hệ thống Auto-bid đưng độ, thuật toán sẽ tự đẩy giá lên giới hạn của người có cấu hình thấp hơn ngay lập tức, đảm bảo logic khắt khe của hệ thống sàn đấu giá thật.

4.2. Hệ thống Chống bắn tỉa phút chót (Anti-Snipe Extension)

Nhằm xóa sổ hành vi dùng tool/bot đánh úp giành giật ở những giây cuối, động cơ của MiniBoom liên tục giám sát tham số thời gian thực. Nếu phát hiện có bất kỳ lệnh đặt giá hợp lệ nào lọt vào mốc 30 giây cuối cùng của phiên, hệ thống sẽ tự động kích hoạt hàm `extendEndTime()`, cộng dồn thêm 60 giây và Broadcast toàn mạng lưới.

4.3. Biểu đồ biến động giá động (Live BidPrice Chart)

Client nhận lệnh `BID_HISTORY_ITEM_ADDED` qua Socket và cập nhật trực tiếp biểu đồ véc-tơ `LineChart` của `JavaFX` thông qua `Platform.runLater()`. Điều này giúp Bidder trực quan hóa được mức độ khốc liệt của phiên đấu giá mà không cần nhấn tải lại danh sách thầu.

4.4. Thanh toán Nguyên tử Hậu kiểm (Atomic Post-Auction Checkout)

Thay vì đóng băng tiền (Hold Balance) cứng nhắc ngay khi người dùng đặt giá, hệ thống áp dụng luồng tiền tự do (eBay model). Khi phiên kết thúc (FINISHED), luồng thanh toán nguyên tử mới được gọi bằng một SQL Transaction: hệ thống đồng thời trừ tiền người mua, cộng tiền người bán và sinh lịch sử giao dịch. Nếu người thắng không đủ tiền, hệ thống tiến hành phạt và hủy phiên (CANCELED), bảo vệ toàn vẹn ACID của nền tảng ảo.